

1. Water Jug Problem

Aim

To obtain exactly 2 gallons of water using a 4-gallon jug and a 3-gallon jug without any measuring device.

Problem Understanding

You are given:

- One 4-gallon jug
- One 3-gallon jug
- An unlimited water source (pump)
- Water can be filled, emptied, or transferred between the jugs.
- No measuring markings are available.

Goal: Measure exactly 2 gallons in the 4-gallon jug.

State Space Representation

Each state is represented as (x, y) where:

- x = amount of water in the 4-gallon jug
- y = amount of water in the 3-gallon jug

Initial State:

$(0,0)$

Goal State:

$(2,0)$

Method Selection

The problem is solved using State Space Search (Breadth-First Search or Depth-First Search).

Reason:

- Each state represents a unique combination of water in the two jugs.
- The search explores all possible states until the goal state is reached.

Algorithm

1. Start with both jugs empty.
2. Fill the 3-gallon jug.
3. Pour water from the 3-gallon jug into the 4-gallon jug.

4. Fill the 3-gallon jug again.
5. Pour water into the 4-gallon jug until it becomes full.
6. Empty the 4-gallon jug.
7. Transfer the remaining water from the 3-gallon jug into the 4-gallon jug.
8. The 4-gallon jug now contains exactly 2 gallons.

Step-by-Step Solution

Step	State (4-Gallon, 3-Gallon)	Action
1	(0,0)	Initial state
2	(0,3)	Fill the 3-gallon jug
3	(3,0)	Pour all water into the 4-gallon jug
4	(3,3)	Fill the 3-gallon jug again
5	(4,2)	Pour water into the 4-gallon jug until it is full (1 gallon remains in the 3-gallon jug)
6	(0,2)	Empty the 4-gallon jug
7	(2,0)	Pour the remaining 2 gallons into the 4-gallon jug

Final Result

The required 2 gallons of water is successfully measured in the 4-gallon jug.

Final State: (2,0)

Conclusion

The Water Jug Problem is a classic Artificial Intelligence state-space search problem. By representing each water level as a state and applying valid operations (fill, empty, and pour), the goal state is reached systematically. This demonstrates how AI search techniques solve problems through logical exploration of possible states.

2. Mars Rover – Analyze and Explore the Samples :

Introduction

A Mars Rover is an autonomous robotic vehicle that explores the surface of Mars. It collects rock and soil samples, captures images, analyzes the environment, and sends scientific data to Earth using Artificial Intelligence.

(i) What are the percepts for this agent?

Percepts are the inputs received by the rover through its sensors.

Percepts include:

- Images of the surroundings
- Rocks and soil
- Obstacles
- Temperature
- Atmospheric conditions
- Battery level
- Distance to objects

These inputs help the rover understand its environment and make decisions.

(ii) Characterize the operating environment

The Mars Rover operates in an environment that is:

- Partially Observable – It cannot observe the entire environment at once.
- Dynamic – Weather and terrain may change.
- Sequential – Current actions affect future actions.
- Continuous – Movement and environmental changes are continuous.
- Unknown – Many regions of Mars are unexplored.

(iii) What are the actions the agent can take?

The rover can perform the following actions:

- Move forward, backward, left, and right
- Avoid obstacles
- Capture images
- Drill rocks
- Collect soil samples

- Analyze samples
- Send data to Earth
- Recharge using solar panels

(iv) How can we evaluate the performance of the agent?

The rover's performance is measured by:

- Number of samples collected
- Accuracy of analysis
- Distance explored
- Battery efficiency
- Successful communication with Earth
- Safe navigation without collisions

A rover that collects more useful data while using less energy has better performance.

(v) What sort of agent architecture is suitable? Why?

Suitable Agent: Utility-Based Learning Agent

Reason:

- Learns from previous experiences.
- Selects the best path to the destination.
- Uses battery power efficiently.
- Adapts to changing terrain and obstacles.
- Maximizes scientific data collection.

Conclusion

The Mars Rover is an intelligent AI agent that senses its environment, makes decisions, performs scientific tasks, and communicates results to Earth. A Utility-Based Learning Agent is the most suitable architecture because it can learn from experience, adapt to changing conditions, and achieve the mission efficiently.

3. 8-Queens Problem using Search Space :

Introduction

The 8-Queens Problem is a classic Artificial Intelligence search problem. The objective is to place 8 queens on an 8×8 chessboard such that no two queens attack each other. A queen can attack another queen in the same row, same column, or same diagonal.

Problem Understanding

Initial State:

- Empty 8×8 chessboard.

Goal State:

- Place all 8 queens so that none of them attack each other.

Constraints:

- Only one queen in each row.
- Only one queen in each column.
- No two queens should be on the same diagonal.

Method Selection

The problem is solved using Backtracking Search Algorithm, which is a search strategy used in AI.

Reason:

- Places queens one row at a time.
- Checks whether the current position is safe.
- If no safe position exists, it backtracks and tries another position.
- Continues until a valid solution is found.

Algorithm

1. Start with an empty chessboard.
2. Place a queen in the first row at a safe position.
3. Move to the next row and place another queen in a safe column.
4. Check that no queens share the same column or diagonal.
5. If a safe position is not available, backtrack to the previous row and move that queen.
6. Repeat the process until all 8 queens are placed safely.

Search Space

The search space consists of all possible arrangements of 8 queens on the chessboard. The algorithm explores these arrangements and eliminates invalid states where queens attack each other.

One Valid Solution

Q

. . . . Q . .

. Q

. Q . .

. . Q

. Q .

. Q

. . . Q . . .

Queen Positions:

- (1,1)
- (2,5)
- (3,8)
- (4,6)
- (5,3)
- (6,7)
- (7,2)
- (8,4)

State Space Diagram

Start (Empty Board)

|

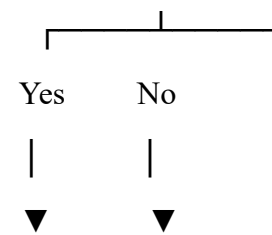


Place Queen in Row 1

|

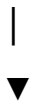


Safe Position?

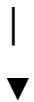


Place Queen Try Next

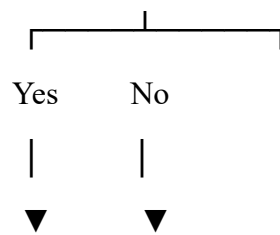
in Row 2 Column



Repeat for Remaining Rows



All 8 Queens Placed?



Solution Backtrack

Result

A valid arrangement of 8 queens is obtained where no queen attacks another. The Backtracking Search algorithm efficiently explores the search space and finds the solution.

Conclusion

The 8-Queens Problem demonstrates the use of state-space search and backtracking in Artificial Intelligence. By checking constraints at every step and backtracking whenever necessary, the algorithm successfully finds a solution without testing every possible arrangement. This makes it an efficient search strategy for constraint satisfaction problems.

4. OLA Cab Booking – Intelligent Agent :

Introduction

OLA Cab Booking is an AI-based transportation system that helps users book a cab to travel from one location to another. The system acts as an intelligent agent by receiving user

requests, finding nearby drivers, selecting the best cab, and completing the booking efficiently.

Problem Understanding

The objective is to book a suitable cab based on the user's:

- Pickup location
- Destination
- Cab type
- Availability of drivers

The agent aims to provide the fastest and most convenient ride.

Suitable Agent

Goal-Based Agent

A Goal-Based Agent is the most suitable because its goal is to successfully book a cab and take the passenger to the destination. It evaluates available options and selects the best one to achieve the goal.

Working of the Agent

1. User enters the pickup and destination locations.
2. The system searches for nearby available cabs.
3. It calculates the estimated fare and arrival time.
4. The user selects a preferred cab.
5. The booking request is sent to the nearest driver.
6. The driver accepts the request.
7. The cab reaches the pickup location.
8. The passenger completes the journey and payment.

PEAS Representation

Component	Description
Performance Measure	Quick booking, minimum waiting time, customer satisfaction, safe journey
Environment	Roads, traffic, passengers, drivers, GPS network
Actuators	Mobile app notifications, booking confirmation, route guidance

Component	Description
Sensors	GPS, user location, traffic data, driver availability

Pseudocode

Start

Enter pickup location

Enter destination

Search for nearby cabs

If cab is available

 Display cab details and fare

 User selects cab

 Send booking request

 Driver accepts booking

 Start trip

Else

 Display "No cabs available"

End

Advantages

- Easy and fast cab booking.
- Reduces waiting time.
- Provides estimated fare and arrival time.
- Uses GPS for accurate navigation.
- Ensures safe and convenient travel.

Conclusion

The OLA Cab Booking System works as a Goal-Based Intelligent Agent because it focuses on achieving the user's goal of reaching the destination safely and efficiently. By using GPS, traffic information, and driver availability, it selects the best cab and provides a smooth travel experience.

5. Uniform Cost Search (UCS) – Elaborate Answer (8–10 Marks)

Introduction

Uniform Cost Search (UCS) is an informed search algorithm used in Artificial Intelligence to find the least-cost path from the start node to the goal node. It always expands the node with the lowest cumulative path cost, ensuring an optimal solution when all edge costs are positive.

Problem Understanding

Given the graph:

- $S \rightarrow A = 1$
- $S \rightarrow G = 12$
- $A \rightarrow B = 3$
- $A \rightarrow C = 1$
- $B \rightarrow D = 3$
- $C \rightarrow D = 1$
- $C \rightarrow G = 2$
- $D \rightarrow G = 3$

Initial State: S

Goal State: G

Objective: Find the minimum-cost path from S to G using Uniform Cost Search.

Method Selection

Algorithm Used: Uniform Cost Search (UCS)

Reason:

- Expands the node with the smallest accumulated cost.
- Guarantees the shortest path when edge costs are positive.

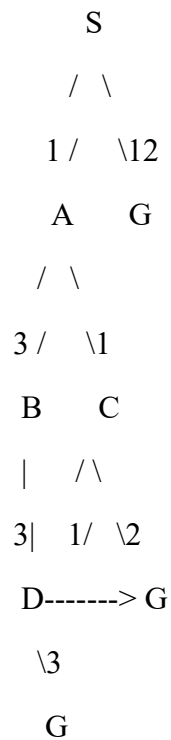
Step-by-Step Solution

Step Expanded Node Path Cost Frontier

1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	D(3), G(4), B(4)
4	D	3	G(4), B(4), G(6)
5	G	4	Goal reached

Since the first goal node removed from the priority queue has the lowest cost (4), the search stops.

Search Tree



Cost Calculation

Possible paths:

- $S \rightarrow G = 12$
- $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 1 + 3 + 3 + 3 = 10$
- $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 1 + 1 + 1 + 3 = 6$
- $S \rightarrow A \rightarrow C \rightarrow G = 1 + 1 + 2 = 4$ ✓

Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Total Cost = 4

Advantages of Uniform Cost Search

- Finds the optimal (minimum-cost) path.
- Works well when edge costs are different.
- Complete if all costs are positive.
- Widely used in pathfinding and navigation problems.

Conclusion

Uniform Cost Search expands nodes based on the lowest cumulative path cost, ensuring that the optimal solution is found. For the given graph, the minimum-cost path is:

$S \rightarrow A \rightarrow C \rightarrow G$

with a total cost of 4, making it the best solution according to the UCS algorithm.